

PitchPulse AI

Multimodal Soccer Tactical Analyst (Hybrid Architecture)

Author: Ibrahim Yasser | **Location:** Mansoura, Dakahliya, Egypt |

Contact: ibrahimelhady305@gmail.com

1. Executive Summary

Building upon foundational computer vision and full-stack development principles, **PitchPulse AI** scales spatial tracking to a complex, multi-agent environment (soccer). The system translates raw pixel coordinates into tactical text and feeds this data into a cloud-based Generative AI reasoning engine. This specific blueprint employs a **Hybrid Cloud/Local Architecture**, utilizing local hardware for edge-vision inference while orchestrating cloud APIs for heavy language and audio processing.

Constrained Hardware Optimization Strategy

Tailored explicitly for environments with strict resource limits (8GB System RAM, Nvidia GeForce MX450 2GB VRAM). Attempting simultaneous local LLM and CV inference on this hardware will cause Out-Of-Memory (OOM) failures. Therefore, this architecture designates the local GPU exclusively for a Nano-tier Vision model (YOLOv8n) and offloads all text-generation and audio-transcription tasks to high-speed cloud APIs (Google Gemini / Deepgram), ensuring smooth, crash-free execution.

2. System Architecture

Module A: The Edge Vision Pipeline (Local Inference)

This module runs entirely on the local MX450 GPU to process video frames without latency.

- **Multi-Object Tracking:** Deploy **YOLOv8 Nano (YOLOv8n)** to detect players, referees, and the ball. The "Nano" variant requires minimal VRAM, leaving room for frame buffering.
- **Dynamic Homography:** Using OpenCV, calculate a homography matrix to transform camera-view pixel coordinates (u, v) into top-down, 2D pitch coordinates (x, y) .

Module B: Semantic Translation Engine (Local Logic)

A lightweight Python script that acts as the bridge, mapping spatial data to soccer terminology before data leaves the local machine.

- **Event Triggers:** Detect state changes. If the ball's (x, y) intersects with Player 10, and rapidly moves to Player 9, log: *"Pass completed from Player 10 to Player 9."*
- **JSON Payload Generation:** Package these events into lightweight JSON structures to be sent over HTTP.

Module C: Generative Analytics Brain (Cloud API Integration)

This module provides the tactical intelligence, removing the need for massive local RAM.

- **API Orchestration:** Instead of local `llama.cpp`, transmit the semantic event JSONs to the **Google Gemini API** or **Groq API**.
- **System Prompts:** Utilize rigorous prompt engineering to instruct the cloud LLM to act as a tactical assistant coach, digesting the JSON and returning insights.

Module D: Multimodal Audio Service (Cloud API)

Managing audio ingestion without overwhelming local CPU threads.

- **Speech-to-Text:** Stream user voice queries to an API like Deepgram or OpenAI Whisper API for near-instantaneous, low-resource transcription.

Module E: The Flask Backend & Real-Time Communications

- **Flask-SocketIO:** Manage the bidirectional WebSocket connections, streaming YOLO coordinates to the frontend and handling the asynchronous callbacks from the LLM APIs.

Module F: Custom Web Application Interface

- **Interactive 2D Pitch Radar:** Utilize HTML5 Canvas (via React) to dynamically render the tactical radar, plotting player dots as data arrives via WebSockets.
- **AI Chat Console:** Display the API-generated tactical insights and commentary.

3. Technology Stack

Domain	Technologies & Frameworks
Computer Vision (Local Edge)	YOLOv8 Nano (YOLOv8n), OpenCV, ByteTrack
Generative AI & Audio (Cloud APIs)	Google Gemini API / Groq API, Deepgram API (Speech)
Data & Backend	Python, Flask, Flask-SocketIO, Requests / aiohttp
Frontend Interface	React.js / Next.js, Tailwind CSS, HTML5 Canvas